

Lab01

Environment

Everything below was run on my own machine, so here is a quick reference so any command in this report can be reproduced exactly.

Shell	<code>zsh</code> (Labs 1-8 and 10), <code>bash</code> (Lab 9)
Working directory	<code>~/Documents/HCMUS/linux</code> = <code>/home/hung/Documents/HCMUS/linux</code>
Log file	<code>lab01/access.log</code> = <code>/home/hung/Documents/HCMUS/linux/lab01/access.log</code>
Log details	17,872,192 bytes (~17 MB), 100,000 lines, nginx "combined" format
Screenshots	<code>screenshots/</code> - the folder sitting next to this file

I stay in `~/Documents/HCMUS/linux` and the log lives in the `lab01/` subfolder, so every command below refers to it as the **relative** path `lab01/access.log`. One sample line:

```
203.0.113.88 - - [22/May/2026:00:00:01 +0700] "GET /static/css/main.css HTTP/1.1" 200 16889 "-"
"python-requests/2.31.0"
```

Lab 1 - Map an unknown directory

Navigation · tree.

Create a nested directory `~/lab01/{src,test,docs}/{img,raw}` in a single command.

```
mkdir -p ~/lab01/{src,test,docs}/{img,raw}
```

Explain: I use `mkdir -p` to create multi-level directories, it will automatically create all parent directories if need to. Also, I use `{...}` for creating multiple directories at the same level. For example, `{1,2}/{3,4}` will create 4 directories: `1/3`, `1/4`, `2/3` and `2/4`

Drop empty files into each leaf folder and verify with `tree -L 3` (or `find . -type f` if `tree` isn't installed).

```
touch ~/lab01/{src,test,docs}/{img,raw}/{hello.txt,hi.txt}
tree ~/lab01
```

Explain: I use `touch` to create multiple files at the same time, `{}/{}` follows the same rule as `mkdir` above.

```
1:code 2:zsh 26-09 22:29
> touch ~/lab01/{src,test,docs}/{img,raw}/{hello.txt,hi.txt}
> tree ~/lab01
/home/hung/lab01
├── docs
│   ├── img
│   │   ├── hello.txt
│   │   └── hi.txt
│   └── raw
│       ├── hello.txt
│       └── hi.txt
├── src
│   ├── img
│   │   ├── hello.txt
│   │   └── hi.txt
│   └── raw
│       ├── hello.txt
│       └── hi.txt
└── test
    ├── img
    │   ├── hello.txt
    │   └── hi.txt
    └── raw
        ├── hello.txt
        └── hi.txt
10 directories, 12 files
~/Documents/HCMUS/linux
>
```

`touch` then `tree ~/lab01` - 10 directories, 12 files.

Count only the directories under `~/lab01` - not the files.

```
find ~/lab01 -type d | wc -l
```

Explain: First, I list all the directories inside and including `~/lab01` itself via `find -type d`. `-type d` means that the `type` is `directory`. Because the output of `find` would be lines of path to distinct

directories, I use pipe `|` to feed the output to `wc -l`, means **w**ord **c**ount `-l` is lines so I can count how many lines there are. Since each directory is on a single line, count lines will show me how many directories there are inside and including `~/lab01`.

Print the full path of every file using one command.

```
find ~/lab01 -type f
```

I use `find` with `-type f` to find all files inside `~/lab01`, it will output all files.

```
1:code 2:zsh 26-09 22:31
> find ~/lab01 -type d | wc -l
10
> find ~/lab01 -type f
/home/hung/lab01/src/img/hello.txt
/home/hung/lab01/src/img/hi.txt
/home/hung/lab01/src/raw/hello.txt
/home/hung/lab01/src/raw/hi.txt
/home/hung/lab01/test/img/hello.txt
/home/hung/lab01/test/img/hi.txt
/home/hung/lab01/test/raw/hello.txt
/home/hung/lab01/test/raw/hi.txt
/home/hung/lab01/docs/img/hello.txt
/home/hung/lab01/docs/img/hi.txt
/home/hung/lab01/docs/raw/hello.txt
/home/hung/lab01/docs/raw/hi.txt
~/Documents/HCMUS/linux
```

`find ~/lab01 -type d | wc -l` gives 10, then `find ~/lab01 -type f` lists all 12 files by full path.

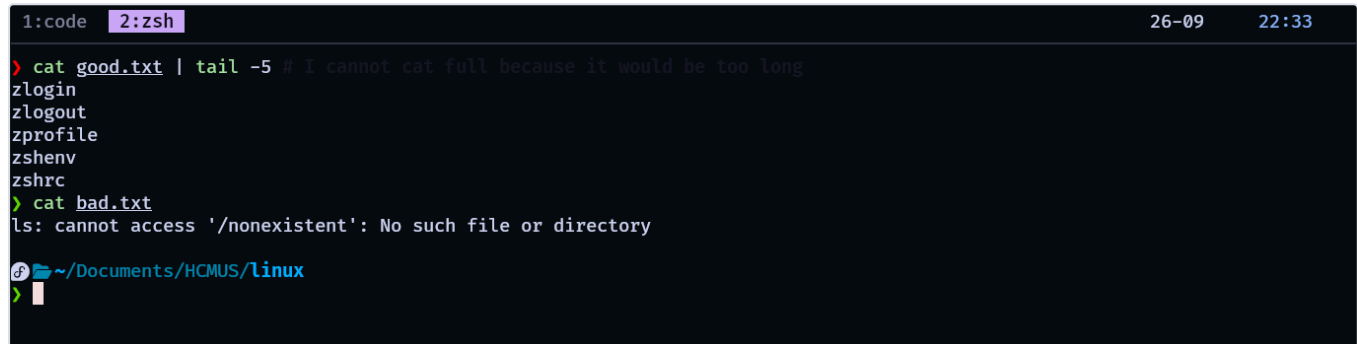
Lab 2 - Stdout vs stderr - split the streams

Tight redirection drill.

Task 1 - Redirect only stdout to `good.txt`, only stderr to `bad.txt`

```
ls /etc /nonexistent 1> good.txt 2> bad.txt
```

Explain: I use `1> good.txt` to redirect the stdout (1) to `good.txt` and `2> bad.txt` to redirect errors to `bad.txt` since errors do not appear in stdout



`cat good.txt | tail -5` lists real files from `/etc`; `cat bad.txt` holds only the error line.

Task 2 - Combine both streams into one file `all.txt`

```
ls /etc /nonexistent > all.txt 2>&1
```

Explain: I use `> all.txt` to redirect stdout(1) to `all.txt`, and `2>&1` means send file descriptor 2 (stderr) to wherever file descriptor 1 (stdout) is currently going, which is a file named `all.txt`.

So order matters, `cmd >f 2>&1` will redirect descriptor 2 (stderr) to what descriptor 1 is currently going, which is `f`. Comparing to `cmd 2>&1 >f`, we first, redirect stderr `2>&1` to what stdout is currently going, which is still stdout, so we just make stderr to go to console, then we use `>f` to redirect stdout to a file `f`, now the stdout is redirected to the file, but the stderr is still at the console.

Task 3 - Discard stderr, keep stdout on screen

```
ls /etc /nonexistent 2> /dev/null
```

Explain: I just use redirect stderr to `/dev/null`, which will not store/show it anywhere.

Task 4 - `tee` sends `uptime` to a file AND the screen

```
uptime | tee file.txt
```

Explain: `uptime` sends its stdout to `tee`'s stdin. `tee` then writes that input both to its `stdout` and to `file.txt`.

```
1:code 2:zsh 26-09 22:34
> uptime | tee file.txt
22:34:32 up 5:43, 2 users, load average: 0.97, 1.17, 1.91
> cat file.txt
22:34:32 up 5:43, 2 users, load average: 0.97, 1.17, 1.91
~/Documents/HCMUS/linux
```

`uptime | tee file.txt` prints on screen, and `cat file.txt` shows the identical line in the file.

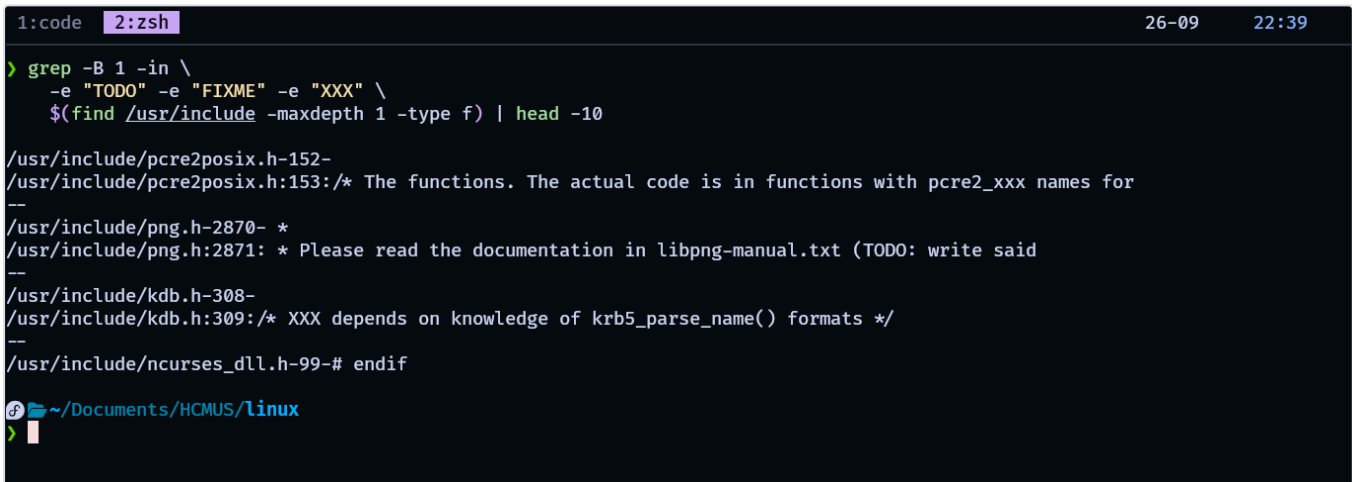
Lab 3 - Find every TODO in a codebase

grep pattern-hunting.

Task 1 - Clone a small repo (or use `/usr/include`); find every line with TODO / FIXME / XXX, case-insensitive, with filename, line number, and 1 line of context above

```
grep -B 1 -in \  
-e "TODO" -e "FIXME" -e "XXX" \  
$(find /usr/include -maxdepth 1 -type f)
```

Explain: I use `grep` with `-B 1` to add 1 line of context above, `-in` to add case insensitive and line number and I use `-e "text"` to indicate the pattern, then I use `$(find /usr/include -maxdepth 1 -type f)` to make it search for all files with maxdepth 1 (because maxdepth 2 or higher would make the output too long to read) inside `/usr/include`, `$` means: use the result of this `find` command as arguments for `grep`.



```
1:code 2:zsh 26-09 22:39  
> grep -B 1 -in \  
-e "TODO" -e "FIXME" -e "XXX" \  
$(find /usr/include -maxdepth 1 -type f) | head -10  
  
/usr/include/pcres2posix.h-152-  
/usr/include/pcres2posix.h:153:/* The functions. The actual code is in functions with pcre2_xxx names for  
--  
/usr/include/png.h-2870- *  
/usr/include/png.h:2871: * Please read the documentation in libpng-manual.txt (TODO: write said  
--  
/usr/include/kdb.h-308-  
/usr/include/kdb.h:309:/* XXX depends on knowledge of krb5_parse_name() formats */  
--  
/usr/include/ncurses_dll.h-99-# endif  
  
~/Documents/HCMUS/linux  
>
```

Ten matches from `/usr/include` - the `-` separator lines are the 1 line of context from `-B 1`.

Task 2 - Exclude `node_modules/`, `.git/`, and binary files; then count how many distinct files contain a TODO

```
grep -lir --exclude-dir=.git --exclude-dir=node_modules \  
-e "TODO" \  
/usr/include | wc -l
```

Explain: I use `-r` for recursion, `exclude-dir=` to exclude directories, then use `-l` to output only distinct file. Also, I pipe the grep output with `wc -l` to count lines as in lab 1 -> get how many files matched.

```
1:code 2:zsh 26-09 22:39
> grep -lir --exclude-dir=.git --exclude-dir=node_modules \
  -e "TODO" \
  /usr/include | wc -l
748
~/Documents/HCMUS/linux 5s
```

`grep -lir ... | wc -l` returns 748 files.

Questions (in report)

- What repo/folder did you search, and roughly how many files? I searched `/usr/include` (the system C headers). I limited it with `find -maxdepth 1 -type f`, so about 226 files. The recursive search in Task 2 found TODO in 748 files.
- Pick one real TODO - what would you fix and why? In `/usr/include/png.h:2871` (libpng) there is `TODO: write said documentation`. I would write that documentation (or remove the TODO once it is done), because an API without documentation is easy to misuse, and that causes bugs for everyone using libpng.

Lab 4 - Top 10 IPs in access.log

awk log analytics on `access.log`.

Where the log is: I run every command in this lab from `~/Documents/HCMUS/linux`, and the file itself lives in the `lab01/` subfolder, so I always refer to it as the relative path `lab01/access.log` (absolute: `/home/hung/Documents/HCMUS/linux/lab01/access.log`, 100,000 lines).

Task 1 - Top 10 IPs by request count (`awk` + `sort` + `uniq`)

```
awk '{print $1}' lab01/access.log | sort | uniq -c | sort -k1 -nr | head -10
```

Explain: I use `awk` to get the IP of all request, then `sort` it and count how many times each individual IP appear via `uniq -c`. Then I sort the output descending by times the IP appears using `sort -k1 -nr` means `sort` by column 1 (count/times) as number but reversed (descending) `-nr`. All piped to `head -10` to get first 10 lines.

```
1:code 2:zsh 26-09 22:40
> awk '{print $1}' lab01/access.log | sort | uniq -c | sort -k1 -nr | head -10
9800 203.0.113.42
7200 198.51.100.17
5400 203.0.113.88
4100 45.155.205.233
3500 172.105.34.12
2900 8.8.8.8
2400 104.131.55.21
2100 185.220.101.7
1850 162.158.74.55
1600 66.249.66.1
~/Documents/HCMUS/linux ✓ 0|0|0|PIPE|0
```

`awk '{print $1}' lab01/access.log | sort | uniq -c | sort -k1 -nr | head -10` - 203.0.113.42 leads with 9800 requests.

Task 2 - Top 5 URLs that returned 404 - what were people hitting?

```
awk '$9 == 404 {print $7}' lab01/access.log | sort | uniq -c | sort -rn | head -5
```

Explain: I use `awk` to check that the status code (field `$9`) is exactly 404, then print the URL (field `$7`). Checking with `$9 == 404` is more exact than `grep "404"`, because `grep` would also match the number 404 inside other fields (like a byte size of 40428). Then I sort, count and rank like in Task 1, and take the top 5 with `head -5`.

```
1:code 2:zsh 26-09 22:40
> awk '$9 == 404 {print $7}' lab01/access.log | sort | uniq -c | sort -rn | head -5
1468 /wp-admin
1277 /wp-login.php
1059 /.env
901 /admin
699 /phpmyadmin
~/Documents/HCMUS/linux
```

```
awk '$9 == 404 {print $7}' ... | head -5 - /wp-admin is the most-hit missing URL.
```

Stretch - sum total bytes served (field 10) with awk's `END` block

```
awk 'BEGIN {sum = 0}
     {sum += $10}
     END {print sum}' \
lab01/access.log
```

Explain: It is like programming, i initialize variable `sum = 0`, then do a for loop-like each line using `awk` each line, I do `sum += $10`, then after compute for the last line, I print the result.

Questions (in report)

- What does the 404 list tell you about the traffic - real users or bots? The top 404 targets are `/wp-admin`, `/wp-login.php`, `/.env`, `/admin` and `/phpmyadmin`. Real users do not request things like `/.env` or `/phpmyadmin`, so this is bot traffic scanning for WordPress sites, exposed config files and database admin panels.
-

Lab 5 - Request-rate report by hour

awk grouping & aggregation.

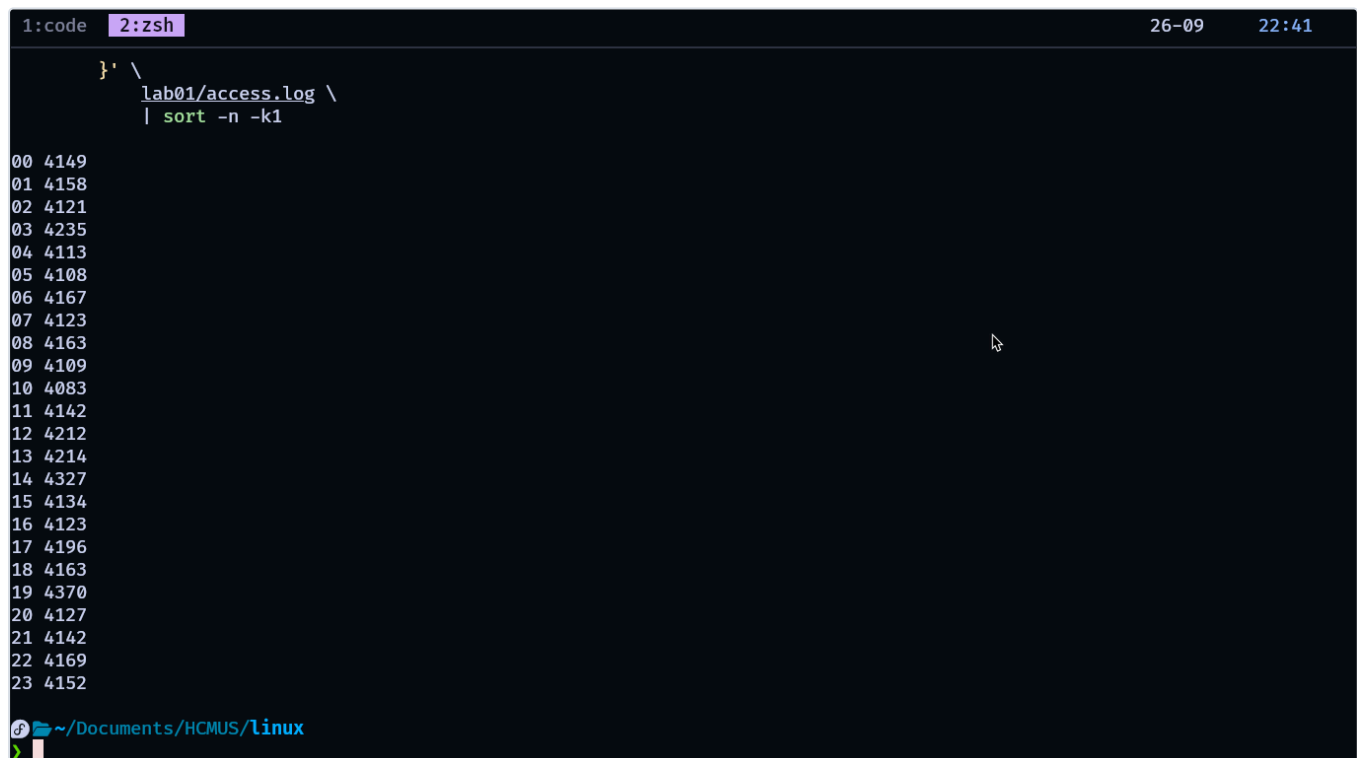
Same `lab01/access.log` as Lab 4, same working directory - see the note at the top of Lab 4.

Hint: `split($4, a, ":")` breaks the timestamp - `a[2]` is the hour. `cnt[hour]++` does the grouping.

Task 1 - Count requests per hour of day, output `HH count` sorted by hour

```
awk ' {
    split($4, a, ":");
    hour = a[2]
    arr[hour]++;
}
END {for(hour in arr)
    print hour " " arr[hour]
}' \
    lab01/access.log \
    | sort -n -k1
```

Explain: I declare an (map) `arr` variable to store result, then for each line, i split to get hour then increase `arr[hour]`, after that, I print the result by iterate through all keys of `arr`. Finally, I print out the result with the right format.



The terminal screenshot shows the command being executed in a zsh shell. The command is an awk script that counts requests per hour from lab01/access.log, followed by a sort command to sort the output by hour. The output shows 24 lines of data, each representing an hour from 00 to 23, with the number of requests for that hour. The output is sorted by hour, and hour 19 is the busiest with 4370 requests.

```
1:code 2:zsh 26-09 22:41
}' \
lab01/access.log \
| sort -n -k1
00 4149
01 4158
02 4121
03 4235
04 4113
05 4108
06 4167
07 4123
08 4163
09 4109
10 4083
11 4142
12 4212
13 4214
14 4327
15 4134
16 4123
17 4196
18 4163
19 4370
20 4127
21 4142
22 4169
23 4152
~/Documents/HCMUS/linux
```

All 24 hours, sorted by hour - hour 19 is the busiest with 4370 requests.

Task 2 - Per HTTP status code: count and share of total traffic (1 decimal, %)

```
awk 'BEGIN {sum = 0}
{
    code = $9;
    codes[code]++;
    sum++;
}
END {for (code in codes) printf "%s %d %.1f%%\n", code, codes[code], codes[code] * 100 /
sum}' \
lab01/access.log
```

Explain: I use `codes` to store all status codes and their count. Also, I use a variable called `sum` to store requests count for calculating the percentage.



The terminal screenshot shows the execution of the awk command in a zsh shell. The command is used to process the lab01/access.log file. The output displays the status code, the count of requests, and the percentage of total traffic for several status codes.

```
1:code 2:zsh 26-09 22:42
> awk 'BEGIN {sum = 0}
{
    code = $9;
    codes[code]++;
    sum++;
}
END {for (code in codes) printf "%s %d %.1f%%\n", code, codes[code], codes[code] * 100 / sum}' \
lab01/access.log
200 88044 88.0%
301 2481 2.5%
403 1002 1.0%
404 6964 7.0%
500 1509 1.5%
```

`200` is 88.0% of traffic, `404` is 7.0% and `500` is only 1.5%.

Task 3 - The single hour with the highest 5xx error count

```
# Find all lines with 5XX status codes
awk '$9 >= 500 && $9 <= 599 {print}' lab01/access.log |
# count and find the highest count
awk '{
    split($4, a, ":");
    hour = a[2];
    arr[hour]++;
}
END {
    for (hour in arr)
        print hour, arr[hour]
}' |
sort -nr -k2 | head -1
```

Explain: First, I use `awk` to filter all lines that have the `5xx` error code, then I piped the output to another `awk`, then calculate the count for each hour. Finally I print the top 1 hour and counts of that

hour.

Stretch - average response size for 200 OK responses only

```
awk '$9 == 200 {sum += $10; line_count++}
END {if (line_count == 0){ line_count = 1;}
    printf "%.2f\n", sum / line_count}' \
lab01/access.log
```

Explain: I choose all line that has 200 as a response code, then I calculate the file size sum, and number of lines. Finally, I print out the result.

Questions (in report)

- What was the peak hour, and what was the worst-5xx hour? Are they the same? What would that mean? The peak hour which received most requests was hour 19 with 4370 request (based on Task 1). But the hour which has most 5XX response was hour 10, not the same as peak hour. So we can conclude that the traffic volume didn't cause the failures, maybe there are other problems.
-

Lab 6 - Bulk-replace strings in config files

sed · find · xargs.

Watch out: `sed -i` edits in place - no undo. Test on one file, or use `-i.bak`.

Task 1 - Create 5 `.conf` files under `~/lab06/configs/` each containing `db_host=old-server.local`

```
mkdir -p ~/lab06/configs

tee ~/lab06/configs/file{1..5}.conf <<'EOF'
db_host=old-server.local
EOF
```

Explain: I create the required directory with `mkdir -p`. Then I use `tee` to write the same content to five files at the same time. `<<'EOF'` tells the shell to use the following lines as input until it reaches `EOF`.

A terminal window with a dark background. The prompt is '1:code' and '2:zsh'. The user has run 'ls ~/lab06/configs' which lists 'file1.conf' through 'file5.conf'. Then they ran 'cat ~/lab06/configs/file1.conf' which outputs 'db_host=old-server.local'. The terminal also shows the path '~/Documents/HCMUS/Linux/Lab01'.

`ls ~/lab06/configs` lists `file1.conf` ... `file5.conf`, and `cat file1.conf` still shows `db_host=old-server.local`.

Task 2 - Replace every `old-server.local` with `10.5.0.2` via a single `find ... | xargs sed -i` pipeline

```
find ~/lab06 -type f -print0 | xargs -0 sed -i 's/old-server\.local/10.5.0.2/'
```

Explain: I use `find` to find all config files, then pipe to `xargs` to make them as arguments of `sed`. Then I use `s/old-server\.local/10.5.0.2/` to substitute all `old-server.local` with `10.5.0.2`. The `\.` escapes the dot, so `sed` matches a literal dot instead of "any character".

Task 3 - Verify all files changed without opening them - use `grep -r`

```
grep -r -e "old-server.local" ~/lab06
```

Explain: After the replacement, no file should contain `old-server.local` anymore, so this `grep -r` prints nothing - and empty output means every file was changed. If I search for `10.5.0.2` instead, I can see the new value in all 5 files.

```
1:code 2:zsh 26-09 22:45
> find ~/lab06 -type f -print0 | xargs -0 sed -i 's/old-server\.local/10.5.0.2/'
> grep -r -e "old-server.local" ~/lab06

~/Documents/HCMUS/linux/lab01
```

The `grep -r` for `old-server.local` prints nothing at all - that empty output is the proof all 5 files changed.

Stretch - before replacing, back each file up to `*.conf.bak` using `sed -i.bak`

```
find ~/lab06 -type f -print0 | xargs -0 sed -i.bak 's/old-server.local/10.5.0.2/'
```

Explain: I add `-i.bak` to make sure `sed` creates a backup before modifying anything.

Questions (in report)

- Why use `xargs` instead of a shell `for` loop?

A `for` loop runs the command once for each item, while `xargs` can batch many items into fewer command executions, making it faster and more efficient for large amounts of input.

- When does `xargs` break

Plain `xargs` can break when filenames contain spaces, tabs, or newlines, because it treats whitespace as separators. Use `xargs -0` with `find -print0` for safe filename handling.

Lab 7 - Disk-hog hunt & permission audit

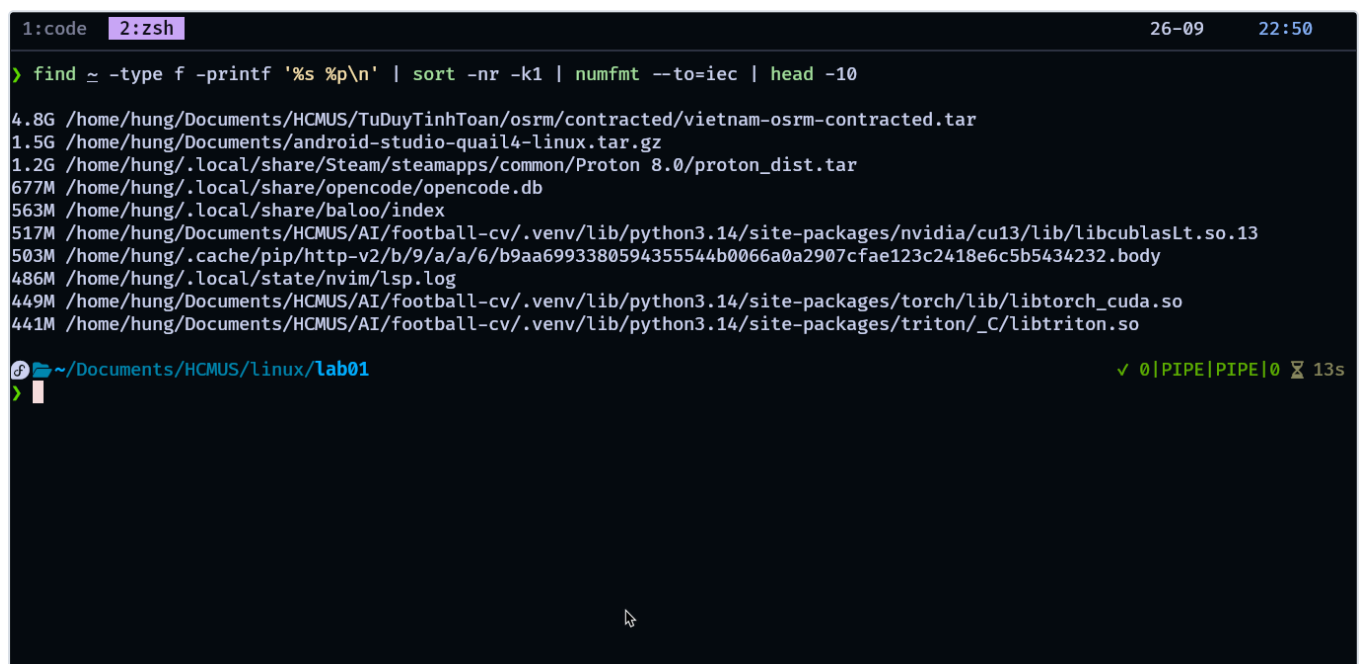
find · permissions · size.

Hint: `find ~ -type f -printf '%s %p\n'` prints size + path → `sort -rn` → `head`. Use `numfmt --to=iec` for human-readable sizes.

Task 1 - The 10 largest files under your home, sorted by size, human-readable

```
find ~ -type f -printf '%s %p\n' | sort -nr -k1 | numfmt --to=iec | head -10
```

Explain: I find all file then print the size and path. Then I sort them descending by first column (size). Finally I pipe them to `numfmt` to make the size human-readable and pipe to `head` to get first 10.



```
1:code 2:zsh 26-09 22:50
> find ~ -type f -printf '%s %p\n' | sort -nr -k1 | numfmt --to=iec | head -10
4.8G /home/hung/Documents/HCMUS/TuDuyTinhToan/osrm/contracted/vietnam-osrm-contracted.tar
1.5G /home/hung/Documents/android-studio-quail4-linux.tar.gz
1.2G /home/hung/.local/share/Steam/steamapps/common/Proton 8.0/proton_dist.tar
677M /home/hung/.local/share/opencode/opencode.db
563M /home/hung/.local/share/baloo/index
517M /home/hung/Documents/HCMUS/AI/football-cv/.venv/lib/python3.14/site-packages/nvidia/cu13/lib/libcublasLt.so.13
503M /home/hung/.cache/pip/http-v2/b/9/a/a/6/b9aa6993380594355544b0066a0a2907cfae123c2418e6c5b5434232.body
486M /home/hung/.local/state/nvim/lsp.log
449M /home/hung/Documents/HCMUS/AI/football-cv/.venv/lib/python3.14/site-packages/torch/lib/libtorch_cuda.so
441M /home/hung/Documents/HCMUS/AI/football-cv/.venv/lib/python3.14/site-packages/triton/_C/libtriton.so
~/Documents/HCMUS/linux/lab01 0|PIPE|PIPE|0 13s
>
```

`find ~ -type f -printf '%s %p\n' | sort -nr -k1 | numfmt --to=iec | head -10` - a 4.8G OSM extract on top.

Task 2 - Every file modified in the last 24 hours under `~/lab06`

```
find ~/lab06 -type f -mtime -1
```

Explain: I use `-mtime -1` to find all files that are modified under 24 hours from now.

Task 3 - Every world-writable file under `/tmp` (`-perm -o+w`) - look, don't change

```
sudo find /tmp -type f -perm -o+w
```

Explain: `-perm -o+w` keeps files where the "other" (`o`) write bit is set, meaning any user on the system can modify them. `-type f` only looks at regular files. On my machine this found nothing, so `/tmp` was clean.

```
1:code 2:zsh 26-09 22:57
> sudo find /tmp -type f -perm -o+w

f ~/Documents/HCMUS/linux/Lab01
>
```

`sudo find /tmp -type f -perm -o+w` - no output, so nothing under /tmp is world-writable.

Stretch - empty files AND empty directories in your home, listed separately

```
find ~ -type f -empty # empty files
find ~ -type d -empty # empty dirs
```

Explain: I use `-empty` instead of `-size 0`, because an empty directory is not size 0 - a directory always takes some space for its metadata (like 40 bytes on ext4), so `-size 0` would miss it. I run two separate commands with `-type f` and `-type d` to list empty files and empty dirs separately.

Questions (in report)

- Why is a world-writable file potentially dangerous? Give one realistic attack scenario. A world-writable file is potentially dangerous because any user or local process on the system can modify, overwrite, or delete its contents.

Example: Assume that we have a file called `backup.sh` that the system would execute every time before an upgrade. But `backup.sh` is a world-writable file so that a person or process can easily change what is inside `backup.sh`. Then maybe next time `backup.sh` is executed, the system would break due to some malicious modification in `backup.sh` by bad people.

Lab 8 - Hard links, soft links, and the inode

inodes & links.

The mental model: a file IS its inode. A directory name is a pointer to one. Hard links add pointers; symlinks are tiny files holding a path string.

Task 1 - Create `original.txt`, plus one hard link and one symlink to it

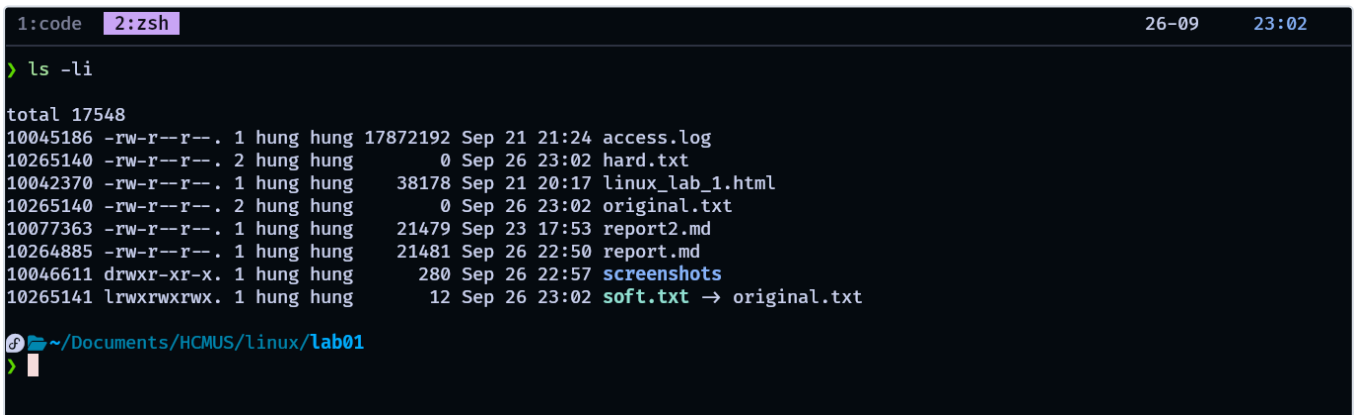
```
touch original.txt
ln original.txt hard.txt
ln -s original.txt soft.txt
```

Explain: I create a hardlink using `ln`, symlink using `ln -s`

Task 2 - Compare inodes with `ls -li` - hard link shares the inode, the symlink doesn't

```
ls -li
```

Explain: Shown in image of `ls -li`, the inode number of `hard.txt` and `original.txt` are the same. But `soft.txt` has a different inode number.



```
1:code 2:zsh 26-09 23:02
> ls -li
total 17548
10045186 -rw-r--r--. 1 hung hung 17872192 Sep 21 21:24 access.log
10265140 -rw-r--r--. 2 hung hung      0 Sep 26 23:02 hard.txt
10042370 -rw-r--r--. 1 hung hung 38178 Sep 21 20:17 linux_lab_1.html
10265140 -rw-r--r--. 2 hung hung      0 Sep 26 23:02 original.txt
10077363 -rw-r--r--. 1 hung hung 21479 Sep 23 17:53 report2.md
10264885 -rw-r--r--. 1 hung hung 21481 Sep 26 22:50 report.md
10046611 drwxr-xr-x. 1 hung hung 280 Sep 26 22:57 screenshots
10265141 lrwxrwxrwx. 1 hung hung 12 Sep 26 23:02 soft.txt -> original.txt
~/Documents/HCMUS/linux/Lab01
>
```

`hard.txt` and `original.txt` are both inode `10265140` with link count 2; `soft.txt` is inode `10265141` and is flagged `-> original.txt`.

Task 3 - Delete the original - which link survives, and why?

```
rm original.txt
cat hard.txt # still works
cat soft.txt # becomes a broken link
```

Explain: When we delete `original.txt`, the hard link `hard.txt` still works, but the soft link `soft.txt` becomes broken.

The reason is that `original.txt` and `hard.txt` point to the same inode. Deleting `original.txt` only removes that filename; the inode and its data are still kept because `hard.txt` still points to the same

inode.

A soft link is different. `soft.txt` has its own inode and stores the path to `original.txt`. When `original.txt` is deleted, that path no longer exists, so `soft.txt` becomes a broken link.

```
1:code 2:zsh 26-09 23:03
> touch original.txt
ln original.txt hard.txt
ln -s original.txt soft.txt

> rm original.txt

> cat hard.txt # still works
> cat soft.txt # becomes a broken link
cat: soft.txt: No such file or directory
~/Documents/HCMUS/linux/Lab01
```

After `rm original.txt`: `cat hard.txt` succeeds silently, while `cat soft.txt` reports "No such file or directory".

Stretch - symlink a file in another dir, then `mv` it to a third location

```
mkdir p1
touch p1/far.txt
ln -s p1/far.txt softfar.txt
```

```
mv p1/far.txt ../far.txt
cat softfar.txt # -> no such file or directory
```

Explain: A symbolic link stores a path. When the target file is moved, the path stored in the symbolic link doesn't change, so the link becomes broken.

Lab 9 - Make your shell yours

`.bashrc` productivity. Attach your `.bashrc` snippet.

Task 1 - Add at least 5 useful aliases to `~/.bashrc`

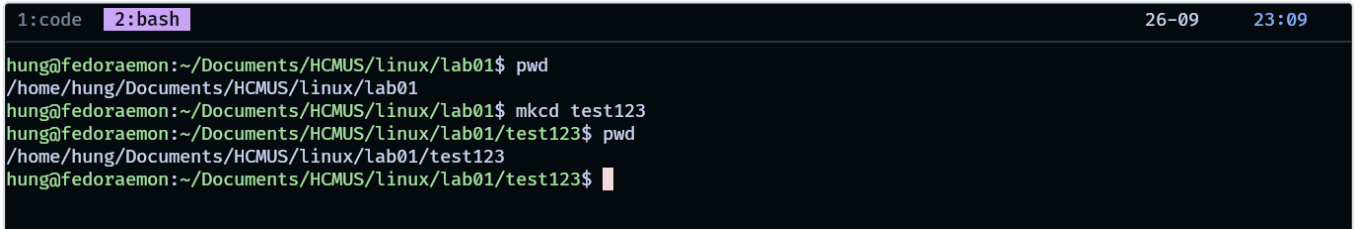
```
alias nvimrc='cd ~/.config/nvim'
alias runcpp='g++ -std=c++23 -Wall -Werror *.cpp -o main && ./main'
alias l='ls -al'
alias li='ls -li'
alias ..='cd ..'
```

Explain: Aliases are shortcuts I put in `~/.bashrc`: `l` and `li` are short forms of `ls -al` and `ls -li`, `..` goes up one directory, `nvimrc` jumps to my Neovim config, and `runcpp` compiles and runs my C++ files in one step. They only work in new shells or after running `source ~/.bashrc`.

Task 2 - Write a `mkcd()` function that creates a dir and `cd`s into it

```
mkcd() {
    mkdir "$1" && cd "$1"
}
```

Explain: This is a function, not an alias, because it runs two commands in order and uses the argument (`$1`). I quote `"$1"` so a folder name with spaces still works, and `&&` makes sure I only `cd` if `mkdir` succeeded.

A terminal window with a dark background. The prompt is 'hung@fedoraemon:~/Documents/HCMUS/linux/lab01\$'. The user enters 'pwd' and the output is '/home/hung/Documents/HCMUS/linux/lab01'. Then the user enters 'mkcd test123' and the prompt changes to 'hung@fedoraemon:~/Documents/HCMUS/linux/lab01/test123\$'. Finally, the user enters 'pwd' and the output is '/home/hung/Documents/HCMUS/linux/lab01/test123'.

`mkcd test123` then `pwd` - the prompt moves from `~/Documents/HCMUS/linux/lab01` to `.../lab01/test123`.

Task 3 - Set `HISTTIMEFORMAT` so `history` shows timestamps

```
HISTTIMEFORMAT="%Y-%m-%d %H:%M:%S || "
```

Explain: `HISTTIMEFORMAT` adds a timestamp to every line shown by `history`. `%Y-%m-%d %H:%M:%S` gives a format like `2026-05-22 14:23:11`. I set it in `~/.bashrc` so it applies to every new shell.

```
1:code 2:bash 26-09 23:11
hung@fedoraemon:~/Documents/HCMUS/linux/lab01$ history | tail -10
219 2026-09-26 23:08:48 || pwd
220 2026-09-26 23:09:01 || cd ..
221 2026-09-26 23:09:05 || rm -rf test123/
222 2026-09-26 23:09:09 || pwd
223 2026-09-26 23:09:17 || mkcd test123
224 2026-09-26 23:09:18 || pwd
225 2026-09-26 23:09:43 || history | tail -10
226 2026-09-26 23:10:13 || cd ..
227 2026-09-26 23:10:15 || rm -rf test123/
228 2026-09-26 23:11:01 || history | tail -10
hung@fedoraemon:~/Documents/HCMUS/linux/lab01$
```

`history | tail -10` - each line is prefixed with `2026-09-26 23:09:17 ||`.

Stretch - add an `extract()` function using a `case` on the filename extension

```
extract() {
    case "$1" in
        *.zip)
            unzip "$1"
            return 0
            ;;
        *.tar.gz)
            tar -xzf "$1"
            return 0
            ;;
        *.tar.bz2)
            tar -xjf "$1"
            return 0
            ;;
        *)
            echo "extract: Unsupported file type: $1" >&2
            return 1
            ;;
    esac
}
```

Explain: `extract()` uses a `case` statement to check the file extension and pick the right unpacking command: `*.zip` runs `unzip`, `*.tar.gz` runs `tar -xzf`, `*.tar.bz2` runs `tar -xjf`. If nothing matches, the `*` case prints an error to stderr (`>&2`) and returns exit code 1.

Lab 10 - The 3 AM pager - find the bad deploy

Capstone · multi-tool, on `access.log` from Lab 4.

Same `lab01/access.log` and same working directory as Lab 4 - see the note at the top of Lab 4.

This pulls every tool together. You're on-call: the Payment API is throwing 500s.

Task 1 - Isolate every POST to any `/api/v1/payment*` URL that returned a 5xx, using only `grep` · `awk` · `sed` · `sort` · `uniq` (one pipeline, no scripts/temp files)

```
grep -e "POST /api/v1/payment" lab01/access.log |
awk '$9 >= 500 && $9 <= 599 {print}'
```

Explain: I filter in two steps: first `grep` keeps only the lines with `POST /api/v1/payment`, then `awk` keeps only those whose status code (field `$9`) is between 500 and 599. This gave 31 failing requests.

Task 2 - From those failures, list unique client IPs, in order of who failed the most

```
grep -e "POST /api/v1/payment" lab01/access.log |
awk '$9 >= 500 && $9 <= 599 {print $1}' | sort | uniq -c | sort -rn -k1
```

Explain: I use the same 31 failures from Task 1, but now `awk '{print $1}'` extracts the client IP. Then the usual `sort | uniq -c | sort -rn -k1` counts each IP and ranks them from most to least failures.

Task 3 - What hour did the failures cluster in? Bucket by hour with `awk`

```
grep -e "POST /api/v1/payment" lab01/access.log |
awk '$9 >= 500 && $9 <= 599 {print}' |
awk ' {
    split($4, a, ":");
    hour = a[2]
    arr[hour]++;
}
END {for(hour in arr)
    print hour " " arr[hour]
}' |
sort -nr -k2
```

Explain: `split($4, a, ":")` takes the hour from the timestamp (`a[2]`), and `arr[hour]++` counts the failures per hour. The failures do not really cluster - the worst hour (23) only has 4. That rules out a big burst at one moment and points to a slow, steady problem.

```
1:code 2:zsh 26-09 23:15
awk ' {
    split($4, a, ":");
    hour = a[2]
    arr[hour]++;
}
END {for(hour in arr)
    print hour " " arr[hour]
}' |
sort -nr -k2
23 4
13 3
01 3
17 2
14 2
10 2
07 2
04 2
02 2
21 1
20 1
19 1
15 1
12 1
09 1
08 1
05 1
00 1
~/Documents/HCMUS/linux
>
```

The hour-bucket, worst first - hour 23 has 4 failures, hour 13 and 01 have 3 each, the rest 2 or 1.

Task 4 - One-liner (pipeline only) outputting IP <tab> failure_count <tab> first_seen <tab> last_seen for the top 5 IPs

```
grep -e "POST /api/v1/payment" lab01/access.log |
awk '$9 >= 500 && $9 <= 599 {print}' |
awk '{
    time = substr($4, 2)
    if (!($1 in first_seen)) {
        first_seen[$1] = time
    }
    last_seen[$1] = time
    count[$1]++
}
END {
    for (key in first_seen) {
        printf "%-15s\t%-8d\t%-25s\t%s\n", key, count[key], first_seen[key], last_seen[key]
    }
}' | sort -nr -k2 | head -5
```

Explain: This awk remembers 3 things per IP: `first_seen` (set the first time the IP appears), `last_seen` (updated every time) and `count`. This only works because `access.log` is sorted by time. `printf` puts a tab (`\t`) between the columns, and `sort -nr -k2 | head -5` keeps the top 5 IPs by failure count.

```
1:code 2:zsh 26-09 23:15
> grep -e "POST /api/v1/payment" lab01/access.log |
awk '$9 ≥ 500 && $9 ≤ 599 {print}' |
awk '{
    time = substr($4, 2)
    if (!($1 in first_seen)) {
        first_seen[$1] = time
    }
    last_seen[$1] = time
    count[$1]++
}
END {
    for (key in first_seen) {
        printf "%-15s\t%-8d\t%-25s\t%s\n", key, count[key], first_seen[key], last_seen[key]
    }
}' | sort -nr -k2 | head -5

198.51.100.17 4 22/May/2026:04:06:59 22/May/2026:15:43:02
96.162.171.44 1 22/May/2026:07:05:07 22/May/2026:07:05:07
93.235.78.112 1 22/May/2026:23:36:59 22/May/2026:23:36:59
8.8.8.8 1 22/May/2026:10:59:23 22/May/2026:10:59:23
76.63.200.44 1 22/May/2026:17:39:04 22/May/2026:17:39:04

~/Documents/HCMUS/linux
```

The one-liner's table - 198.51.100.17 has 4 failures between 04:06:59 and 15:43:02; every other IP has exactly 1.

Questions (in report)

- Write-up: 5–10 sentences - what you found and what you'd do next (rollback? rate limit? page the backend team?). What did the data rule out?

I found 31 POST requests to `/api/v1/payment*` that returned a 500 error. All of them are plain 500 errors, no 502 or 503. The top offender is IP 198.51.100.17, with 4 failures in about 11 hours, using a python-requests user agent. The errors do not cluster - the worst hour only has 4, so a load spike is ruled out. The peak traffic hour (19) also had zero payment errors, so volume did not cause this. It looks like one flaky client or bot retrying the same payment path, or maybe an edge case in the API. Next I would rate-limit that IP and check the backend logs for the same window. I would also verify the endpoint is idempotent, and only roll back if a deploy landed before 04:06.